

C++ and Physics Notes for Project 2

Boris pusher, mesh coupling, and the code in `project2/`

Contents

1	Where the numbers B0, 0,0,0 and 0,0,B0 come from	1
1.1	What is B0?	1
1.2	Why <code>Vec3{0.0, 0.0, 0.0}</code> and <code>Vec3{0.0, 0.0, B0}</code> ?	1
1.3	The strange syntax: <code>lambdas</code>	2
2	The physical quantities: q, m, $\mathbf{v}$, w and ω_c	2
2.1	What is ω_c ?	2
2.2	Why <code>steps_per_period = 1000</code> ?	2
3	<code>static_cast<...></code>	3
4	References: why <code>&</code> in the Boris pusher signature?	3
4.1	Why references?	4
5	Creating the grid and filling B	4
5.1	Line 1: calling the constructor	4
5.2	Lines 2–4: range-based <code>for</code>	4
6	Random particles: <code>std::mt19937</code> and distributions	5
7	The interpolation code in detail	5
7.1	<code>bilinear_weights</code> : the shape function S	5
7.2	<code>deposit_moments</code> : particles $\rightarrow$ grid	6
7.3	<code>gather</code> : grid $\rightarrow$ particle	6
7.4	Why the same weights for deposit and gather?	7
7.5	Why the “linear field” test works	7
8	The main function in detail	7
8.1	Overall structure	7
8.2	<code>run_step1_demo</code> : one particle in a uniform B	7
8.3	<code>run_step2_demo</code> : N particles and the mesh	8

1 Where the numbers B0, 0,0,0 and 0,0,B0 come from

In Step 1 we test the Boris pusher in the simplest field configuration that has an exact analytical answer. The code that defines the fields is:

```
const double B0 = 1.0;
const auto E_of = [](double, double) { return Vec3{0.0, 0.0, 0.0}; };
const auto B_of = [B0](double, double) { return Vec3{0.0, 0.0, B0}; };
```

1.1 What is B0?

B0 is simply a *named constant* holding the strength of the magnetic field. Giving it a name (instead of writing 1.0 everywhere) means that if you want to try $B_0 = 2$, you change one line and the formulas for ω_c , the period and the Larmor radius all follow automatically. The “0” in the name is a physics convention: B_0 means “the uniform background value of B ”.

1.2 Why Vec3{0.0, 0.0, 0.0} and Vec3{0.0, 0.0, B0}?

A `Vec3` has three components (x, y, z) . Curly braces fill them in order:

Code	Meaning	Physics
<code>Vec3{0.0, 0.0, 0.0}</code>	$(0, 0, 0)$	$\mathbf{E} = 0$: no electric field
<code>Vec3{0.0, 0.0, B0}</code>	$(0, 0, B_0)$	$\mathbf{B} = B_0 \hat{\mathbf{z}}$: field pointing out of the plane

We choose $\mathbf{E} = 0$ and $\mathbf{B} = B_0 \hat{\mathbf{z}}$ because:

- The particle moves in the (x, y) plane. A field along z is perpendicular to that plane, so $\mathbf{v} \times \mathbf{B}$ lies in the plane and bends the trajectory into a **circle**.
- A pure magnetic force does no work, so the speed $|\mathbf{v}|$ must stay *exactly* constant. That is a very sharp test for the Boris pusher.
- The circle radius and period are known in closed form (see Section 2), so we can compare the simulation with theory.

1.3 The strange syntax: lambdas

`E_of` and `B_of` are not numbers, they are **functions** stored in variables. This is a *lambda expression*:

```
[capture](parameters) { body }
```

- `[B0]` is the *capture list*: it copies the variable `B0` into the lambda so the body may use it. `E_of` needs nothing from outside, so its capture list is empty: `[]`.
- `(double, double)` are the parameters (x, y) , the position where the field is evaluated. They have **no names** because the field is uniform: it does not depend on position, so we never use them. (Unused named parameters would trigger a compiler warning with `-Wall -Wextra`.)
- The body returns the field vector at that position.

Later, `advance_particle` calls `E_of(p.x, p.y)` exactly like a normal function. Making the field a function means that in the future we can plug in a position-dependent field (e.g. a wave) without touching the pusher.

2 The physical quantities: q , m , $\mathbf{v}$, w and ω_c

You guessed correctly: in the `Particle` struct, `q` is the electric charge, `m` the mass, `v` the velocity (a `Vec3`), and `x`, `y` the position. There is also `w`, the *statistical weight* w_p of the macro-particle: one simulated particle stands for many real ions, and w_p says how many (slide 5 of the course).

2.1 What is ω_c ?

ω_c is the **cyclotron (gyro) frequency**: how fast a charged particle rotates around a magnetic field line. Starting from the Lorentz force with $\mathbf{E} = 0$,

$$m \frac{d\mathbf{v}}{dt} = q \mathbf{v} \times \mathbf{B}, \quad \mathbf{B} = B_0 \hat{\mathbf{z}}$$

the components give $\dot{v}_x = \omega_c v_y$, $\dot{v}_y = -\omega_c v_x$ with

$$\omega_c = \frac{qB_0}{m}$$

The solution is a rotation at angular frequency ω_c . Two consequences that the code uses:

$$T = \frac{2\pi}{\omega_c} \quad (\text{period}), \quad r_L = \frac{|\mathbf{v}_\perp|}{\omega_c} \quad (\text{Larmor radius}).$$

In the code, `omega_c = p.q * B0 / p.m` and `period = 2.0 * std::numbers::pi / omega_c`. With $q = m = B_0 = 1$ we get $\omega_c = 1$, $T = 2\pi \approx 6.283$, and since $v = 1$ the radius is $r_L = 1$.

2.2 Why `steps_per_period = 1000`?

The time step is defined from the period: `dt = period / steps_per_period`. So “1000” means: *cut one gyration into 1000 small steps*. Then each step rotates the velocity by about $\omega_c \Delta t = 2\pi/1000 \approx 0.0063$ rad (about 0.36°).

Why this number? It is a **resolution choice**, not a magic constant:

- The Boris rotation conserves $|\mathbf{v}|$ exactly for *any* Δt , but the rotation angle per step is not exactly $\omega_c \Delta t$; it is $\theta = 2 \arctan(\omega_c \Delta t / 2) \approx \omega_c \Delta t [1 - (\omega_c \Delta t)^2 / 12]$. The phase (frequency) error therefore shrinks like Δt^2 .
- Too few steps per gyration (say 5) $\Rightarrow$ visibly wrong orbit. Real production codes typically use a few tens of steps per gyration. We use 1000 (demo) and 2000 (test) because they are cheap here and make the “particle returns to its starting point” check very tight.
- Too many steps just wastes computing time (and accumulates round-off).

You can experiment: change 1000 to 20 in `main.cpp` and watch the final position drift further from the origin, while the final speed stays exactly 1.

3 `static_cast<...>`

A C++ variable has a fixed type (`int`, `double`, `std::size_t`, ...). `static_cast<T>(value)` **explicitly converts** `value` to type `T`, checked by the compiler at compile time. We use it in two places.

In `bilinear_weights`

```
int i0 = static_cast<int>(std::floor((x - g.x0) / g.dx));
```

- $(x - x_0)/\Delta x$ is a `double`, say 2.3 (the particle sits 2.3 cells from the origin).
- `std::floor` rounds down: 2.0 (still a `double`).
- An array index must be an integer, so `static_cast<int>` turns 2.0 into the integer 2. That is the index of the node just left of the particle.

Using `floor` before the cast matters for negative values: a plain conversion to `int` *truncates toward zero* ($-0.3 \rightarrow 0$) while `floor` gives -1 , which is the correct cell.

In the Grid2D constructor

```
n(static_cast<std::size_t>(nx_) * ny_, 0.0),
```

`std::vector` sizes are of the *unsigned* type `std::size_t`, but `nx_` and `ny_` are (signed) `int`. Casting the first factor makes the whole product unsigned, which avoids a signed/unsigned mismatch warning and possible integer overflow on large grids.

Why not the old C-style cast `(int)x`?

`static_cast` is searchable, states your intent, and refuses conversions that make no sense (e.g. turning an unrelated pointer into a number). C++ style prefers it.

4 References: why `&` in the Boris pusher signature?

The signature is

```
void boris_push(Particle& p, const Vec3& E, const Vec3& B, double dt);
```

An important clarification: the `&` here is **not** “taking an address”. In C++ the symbol has two different meanings depending on where it appears:

In a <i>declaration</i> (<code>Particle& p</code>)	a reference : a second name for an existing object
In an <i>expression</i> (<code>&p</code>)	the address-of operator (gives a pointer)

4.1 Why references?

By default C++ passes arguments **by value**, i.e. it makes a copy:

```
void f(Particle p) { p.v = Vec3{}; } // modifies a COPY, caller sees nothing
void g(Particle& p) { p.v = Vec3{}; } // modifies the caller's particle itself
```

- `Particle& p` (non-const reference): the function must **modify** the caller’s particle (update its velocity). Without `&` the update would be applied to a throw-away copy and lost.
- `const Vec3& E` (const reference): the function only needs to **read** `E`. Passing by reference avoids copying 24 bytes on every call, and `const` makes the compiler forbid accidental modification. For a small `Vec3` the speed gain is minor, but it is the standard habit for anything larger than a couple of numbers (think of a `std::vector` with a million elements: copying it would be a disaster).
- `double dt`: a single number is cheap to copy, so it is passed by value.

Rule of thumb: *small built-in types by value; larger objects by `const &`; objects you want to modify by non-const `&`.*

5 Creating the grid and filling B

```
Grid2D grid(21, 21, 1.0, 1.0, 0.0, 0.0);
for (Vec3& B : grid.B) {
    B = Vec3{0.0, 0.0, 1.0};
}
```

5.1 Line 1: calling the constructor

This creates an object `grid` of type `Grid2D` and calls its **constructor** with the arguments in order:

Argument	Parameter
21, 21	<code>nx, ny</code> : number of grid <i>nodes</i> in <i>x</i> and <i>y</i>
1.0, 1.0	<code>dx, dy</code> : spacing between nodes
0.0, 0.0	<code>x0, y0</code> : coordinates of the first node (origin)

21 nodes with spacing 1 give 20 cells, so the domain is $[0, 20] \times [0, 20]$. The constructor’s initializer list allocates the arrays `n`, `v`, `E`, `B`, each with $21 \times 21 = 441$ entries (all zeros initially).

5.2 Lines 2–4: range-based for

`for (Vec3& B : grid.B)` reads “for each element of the array `grid.B`, call it `B`”. It is shorthand for

```
for (std::size_t k = 0; k < grid.B.size(); ++k) {  
    Vec3& B = grid.B[k];  
    ...  
}
```

The `&` is essential: it makes `B` a *reference to the element inside the grid*. Without it (`Vec3 B`) each element would be copied, you would modify the copy, and the grid would stay all zeros.

The body assigns (0,0,1) to every node: a **uniform** $B_z = 1$ everywhere. (The loop variable is named `B` and the member is also `grid.B`. That is legal, they are different things, but it can be confusing.)

6 Random particles: `std::mt19937` and distributions

```
std::mt19937 rng(42);  
std::uniform_real_distribution<double> pos_dist(2.0, 18.0);  
std::uniform_real_distribution<double> vel_dist(-0.5, 0.5);
```

C++ separates two ideas: an **engine** that produces raw random bits and a **distribution** that shapes them into the numbers you want.

- `std::mt19937 rng(42)`: the **Mersenne Twister** engine, a high-quality pseudo-random generator (period $2^{19937} - 1$). The 42 is the **seed**. The sequence is *deterministic*: the same seed always produces exactly the same particles, so runs are reproducible, which is what you want when debugging or testing. Change the seed to get a different (still reproducible) set.
- `std::uniform_real_distribution<double> pos_dist(2.0, 18.0)`: an object that turns engine output into a `double` uniformly distributed in $[2, 18)$. The `<double>` is a *template parameter* choosing the number type.
- You draw a number with `pos_dist(rng)`: the distribution “calls” the engine. Every call yields a new number. That is why the code calls it separately for `p.x` and for `p.y`.

Why [2, 18] for positions? The grid covers $[0, 20]$. Keeping particles away from the edges leaves room for them to move during the 100 steps ($t = 1$, speed at most ≈ 0.71 , so displacement < 1) without leaving the domain. This matters because `bilinear_weights` clamps the cell index; a particle outside the grid would not raise an error but be *extrapolated* from the edge cell (weights outside $[0, 1]$, possibly negative). We do not have boundary conditions yet; they are a later step.

Why $[-0.5, 0.5]$ for velocities? It gives a small random thermal-like spread in both directions. Only `v.x` and `v.y` are randomized; `v.z` is set to 0.

7 The interpolation code in detail

Interpolation is the bridge between the two worlds of a PIC code: particles live at *arbitrary* positions, fields and moments live only at *grid nodes*. The three functions in `interpolation.cpp` are `bilinear_weights`, `deposit_moments` and `gather`.

7.1 bilinear_weights: the shape function S

```
int i0 = static_cast<int>(std::floor((x - g.x0) / g.dx));
int j0 = static_cast<int>(std::floor((y - g.y0) / g.dy));
i0 = std::clamp(i0, 0, g.nx - 2);
j0 = std::clamp(j0, 0, g.ny - 2);

const double fx = (x - g.node_x(i0)) / g.dx;
const double fy = (y - g.node_y(j0)) / g.dy;

cw.w00 = (1.0 - fx) * (1.0 - fy);
cw.w10 = fx * (1.0 - fy);
cw.w01 = (1.0 - fx) * fy;
cw.w11 = fx * fy;
```

1. **Find the cell.** `i0`, `j0` are the indices of the node at the lower-left corner of the cell containing the particle. The other three corners are $(i_0 + 1, j_0)$, $(i_0, j_0 + 1)$, $(i_0 + 1, j_0 + 1)$.
2. **Clamp.** `std::clamp(i0, 0, nx-2)` forces i_0 into $[0, n_x - 2]$ so that $i_0 + 1$ is always a valid index. This prevents out-of-bounds memory access; it does not fix particles that are truly outside the domain (see above).
3. **Fractional position.** $f_x \in [0, 1]$ is how far the particle is between the left node ($f_x = 0$) and the right node ($f_x = 1$); same for f_y .
4. **Weights.** The weight of a corner is the product of the 1D linear weights. This is exactly the slide's order-1 formula $n_i = (1 - \frac{x_p - x_i}{\Delta x})w_p$, $n_{i+1} = \frac{x_p - x_i}{\Delta x}w_p$, applied in x and y . Geometrically, each weight is the *area of the opposite rectangle*: the closer the particle is to a corner, the larger that corner's weight.

The four weights always sum to 1: $(1 - f_x)(1 - f_y) + f_x(1 - f_y) + (1 - f_x)f_y + f_xf_y = 1$.

Worked example. $\Delta x = \Delta y = 1$, origin $(0, 0)$, particle at $(2.3, 5.6)$:

Quantity	Value	Comment
i_0, j_0	2, 5	lower-left node is (2, 5)
f_x, f_y	0.3, 0.6	
w00	$0.7 \times 0.4 = 0.28$	node (2, 5)
w10	$0.3 \times 0.4 = 0.12$	node (3, 5)
w01	$0.7 \times 0.6 = 0.42$	node (2, 6), closest in y
w11	$0.3 \times 0.6 = 0.18$	node (3, 6)
Sum	1.00	

7.2 deposit_moments: particles $\rightarrow$ grid

Implements $n_{ij} = \sum_p S(\mathbf{r}_p - \mathbf{r}_{ij}) w_p$ and $\mathbf{v}_{ij} = \frac{1}{n_{ij}} \sum_p \mathbf{v}_p S(\mathbf{r}_p - \mathbf{r}_{ij}) w_p$.

1. Zero the arrays **n** and **v** (`std::fill`) so that deposits from the previous step do not accumulate.
2. For each particle: get its four weights and four node indices (`g.index(i,j) = j * n_x + i` maps 2D indices to the 1D array), then **add weight * p.w** to **n** at each corner. The velocity array accumulates **weight * p.w * p.v** (a weighted *sum* of velocities).
3. After all particles are deposited, divide each node's velocity sum by its **n**. This turns the sum into a weighted *average*: the bulk velocity. The test `if (g.n[k] > 0.0)` skips empty nodes to avoid dividing by zero.

Caveat in the current code. **n** is the sum of weights per node; it is *not* divided by the cell area $\Delta x \Delta y$, so it is a “weight per node” rather than a true density. The weight-conservation check multiplies by $\Delta x \Delta y$ and passes only because $\Delta x = \Delta y = 1$. If you use another spacing, either divide by the cell area in `deposit_moments` or drop that factor in the check.

7.3 gather: grid $\rightarrow$ particle

```
result = result + cw.w00 * field[g.index(cw.i0, cw.j0)];
result = result + cw.w10 * field[g.index(cw.i0 + 1, cw.j0)];
...
```

It computes $\mathbf{Q}(\mathbf{r}_p) = \sum_{ij} Q_{ij} S(\mathbf{r}_p - \mathbf{r}_{ij})$ (slide 6): the field at the particle is the weighted average of the field at the four surrounding nodes. Close to a corner, that corner counts more.

7.4 Why the same weights for deposit and gather?

Both functions call the *same* `bilinear_weights`. If deposit and gather used different shape functions, a particle would exert a spurious force on itself (*self-force*) and momentum would not be conserved. Using the same S in both directions is a standard PIC requirement.

7.5 Why the “linear field” test works

Bilinear interpolation reproduces *any* function $a + bx + cy$ exactly, so gathering such a field must return the exact value at any position. The test checks this to 10^{-9} ; a bug in the indices or weights would break it immediately.

8 The main function in detail

8.1 Overall structure

```
namespace {  
void run_step1_demo() { ... }  
void run_step2_demo() { ... }  
} // namespace  
  
int main() {  
    std::cout << "kinetic_fisher: ...\n\n";  
    run_step1_demo();  
    run_step2_demo();  
    return 0;  
}
```

- `namespace { ... }` (an *anonymous namespace*) makes the two helper functions private to this file: no other file can call or clash with them.
- `main` is the entry point. It prints a title, runs the two demos in order, and `return 0` tells the operating system “success”.

8.2 `run_step1_demo`: one particle in a uniform B

1. Define the fields (`E_of`, `B_of`, Section 1).
2. Create the particle: `Particle p`; uses the default member values ($q = m = w = 1$, position $(0,0)$), then `p.v = Vec3{1.0, 0.0, 0.0}` gives it speed 1 along x .
3. Compute `omega_c`, `period`, and `dt = period/1000`.
4. Store `speed0 = norm(p.v)` to compare with later.
5. Loop 1000 times calling `advance_particle(p, E_of, B_of, dt)`. Each call performs half a position step, samples the fields, applies `boris_push`, and does the second half position step. Because `p` is passed by reference, it is updated in place.
6. Print the initial and final speed (must be equal) and the final position (must be $\approx (0,0)$ because one full period brings the particle back to its start). The tiny residual, about 2×10^{-5} , is the Δt^2 phase error explained in Section 2.

`std::fixed` and `std::setprecision(6)` just format the printed numbers with six decimals.

8.3 `run_step2_demo`: N particles and the mesh

1. **Grid.** `Grid2D grid(21,21,1,1,0,0)` and fill $\mathbf{B} = (0,0,1)$ at every node (Section 5). `grid.E` stays zero.
2. **Particles.** `std::vector<Particle> particles(N)` creates $N = 200$ default particles. The loop `for (Particle& p : particles)` (reference!) gives each a random position and velocity (Section 6), charge, mass and weight 1, and accumulates `total_weight (= 200)`.
3. **Time loop.** 100 calls of `advance_particles(particles, grid, dt)` with $\Delta t = 0.01$. In each call, for every particle: half position step, **gather** $\mathbf{E}, \mathbf{B}$ from the grid at that position, Boris push, second half position step. Since $\mathbf{B}$ is uniform, the gather returns $(0,0,1)$ everywhere, so every particle gyrates just as in Step 1.

4. **Deposit.** `deposit_moments(grid, particles)` fills `grid.n` and `grid.v` from the final particle positions.
5. **Check.** Sum `n_ij` over the grid (`for (double n_ij : grid.n):` no `&` needed since we only read a number) and compare with `total_weight`. They must agree, because each particle's four weights sum to 1: no weight is lost or created.

Summary of C++ ideas used: lambdas (`[]()`), references (`&`, `const &`), `static_cast`, range-based `for`, constructors and initializer lists, `std::vector`, `<random>` engines and distributions, `<numbers>` constants, and anonymous namespaces.